

Relazione progetto MapReduce

Enrico Melis - 693626

Indice

1	Introduzione	2
2	Architettura generale del framework	2
3	Interfaccia pubblica	2
4	Organizzazione dei processi	3
5	Organizzazione dei thread	4
5.1	Thread del mapper	4
5.2	Thread del reducer	4
5.3	Altri thread	4
6	Code interne e sincronizzazione	4
7	Protocollo sulle pipe	5
7.1	Gestione dell'ownership	5
8	Raggruppamento per token	6
9	Formato del file di output	6
10	Sistema di log	7
11	Esempio applicativo: word count	7
12	Test automatici	7
12.1	Test su attributi e ciclo di vita	8
12.2	Test end-to-end generico	8
12.3	Test su directory, concorrenza e log	8
13	Utilizzo di coding agents nel progetto	8
14	Conclusioni	9

1 Introduzione

La maggior parte della relazione si trova in questo documento, i commenti all'interno del codice sono minimali e spesso limitati alla descrizione delle convenzioni per i valori di ritorno.

2 Architettura generale del framework

Il framework è una libreria C esposta grazie all'header `mr.h`. Si occupa di tutta la gestione della computazione, dalla lettura dell'input alla scrittura dell'output passando per la gestione dei processi, dei dati grezzi e della comunicazione fra processi.

L'utente fornisce le due funzioni chiave: la `mapper` e la `reducer` che vengono usate dal framework per svolgere la computazione richiesta secondo il flusso di dati orchestrato dal framework.

Il flusso avviene tramite tre processi: il main, il mapper e il reducer che comunicano con tre pipe secondo il seguente ordine:

```
file input
-> processo principale
-> pipe righe
-> processo mapper
-> pipe coppie <token, valore>
-> processo reducer
-> pipe risultati
-> processo principale
-> file output
```

I due processi figli sono multithread e il numero dei thread fa parte degli attributi impostabili dall'utente.

3 Interfaccia pubblica

L'interfaccia pubblica è quella parte del framework esposta all'utente e comprende:

- `mr_t`: Handle opaco per una computazione. Contiene gli attributi, le due callback e un puntatore ad eventuali argomenti forniti dall'utente.
- `mr_attr_t`: Collezione di attributi necessari per la computazione. Contiene il numero di worker threads per i processi, la grandezza della coda e il nome del file di log.
- `mr_file_line_t`: Descrittore logico di una riga del file input.
- `mr_value_t`: Descrittore logico di un valore, collezionato come coppia di dati grezzi e grandezza.
- `mr_emit_pair_t`: Firma per la funzione di emissione delle coppie `<token, value>`.
- `mr_emit_result_t`: Firma per la funzione di emissione delle coppie `<token, result>`.
- `mr_mapper_t`: Firma per la callback mapper.
- `mr_reducer_t`: Firma per la callback reducer.
- `mr_attr_init`: Inizializzatore della memoria degli attributi.
- `mr_attr_destroy`: Distruttore della memoria degli attributi.

- `mr_attr_set_mapper_threads`: Setter per il numero di thread worker del processo mapper.
- `mr_attr_set_reducer_threads`: Setter per il numero di thread worker del processo reducer.
- `mr_attr_set_queue_size`: Setter per il numero di slot nella coda ad anello del processo mapper.
- `mr_attr_set_log_file`: Setter per il file di log.
- `mr_create`: Funzione per istanziare la computazione.
- `mr_start`: Funzione per iniziare la computazione.
- `mr_destroy`: Distruttore della memoria dell'handle.

L'interfaccia separa la logica applicativa dalla gestione concorrente: il programma utente fornisce callback e parametri, mentre il framework si occupa dell'esecuzione, della comunicazione tra processi e della produzione dell'output.

4 Organizzazione dei processi

I processi sono organizzati in una pipeline UNIX a tre stadi. Dal processo main si mandano dati al processo mapper, che li manda al processo reducer, che manda i risultati al processo main (che li scrive sul file in output).

Tutti i processi comunicano tramite pipe. Non vengono usati segnali speciali, alla fine del lavoro vengono chiusi i descrittori e viene propagato l'EOF per passare al prossimo stadio di computazione.

- La pipe `main->mapper` trasmette le righe serializzate dopo che sono state lette in input.
- La pipe `mapper->reducer` trasmette le coppie `<token, valore>`.
- La pipe `reducer->main` trasmette i risultati finali.

I processi figli vengono creati con `fork()` e la redirectione avviene tramite `dup2()` per collegare gli standard input/output ai lati corretti delle pipe. Il processo main è ovviamente incaricato di fare `waitpid()` per i figli.

Di seguito i ruoli dei singoli processi:

- Main: `mr_start()` coordina la computazione; `write_input_path_lines()` legge il path di input e invia le righe; `write_reducer_results()` legge i risultati dalla pipe e produce il file di output.
- Mapper: `mapper_process_main()` inizializza la coda interna, crea il thread reader e i worker thread, applicando la callback mapper alle righe ricevute.
- Reducer: `reducer_process_main()` raccoglie tutte le coppie prodotte dal mapper, le raggruppa per token, crea i worker thread reducer e scrive i risultati finali verso il main.

5 Organizzazione dei thread

I thread dei processi sono tutti gestiti tramite la libreria `<threads.h>`, insieme a lock `mtx_t` e condition variables `cnd_t`.

Ogni processo nasce con un thread iniziale di esecuzione. Nel processo principale questo thread coordina la pipeline, invia le righe al mapper, raccoglie i risultati e attende i figli. Nei processi mapper e reducer il thread iniziale si occupa di creare e attendere i thread di lavoro necessari alla computazione.

5.1 Thread del mapper

Il mapper ha:

- un thread reader (esplicito) che si occupa di leggere le righe serializzate e aggiungerle alla coda;
- `mr->attr.mapper_threads` thread per l'elaborazione della callback mapper.

5.2 Thread del reducer

Il reducer usa il proprio thread iniziale per leggere le coppie e raggrupparle in base ai token. Dopo il raggruppamento vengono creati:

- `mr->attr.reducer_threads` thread per l'elaborazione della callback reducer.

5.3 Altri thread

Oltre ai thread principali, all'interno del processo main vengono creati anche dei thread per la gestione dei log, che vengono approfonditi nella sezione dedicata.

6 Code interne e sincronizzazione

Il framework utilizza una coda circolare per la separazione del thread reader dai thread worker nel processo mapper. È descritta dalla struct `mr_line_queue_t` ed è dotata delle sue funzioni per la gestione interna.

È ottimizzata per il pattern producer-consumer con due condition variables dedicate e un lock per la sincronizzazione. Grazie alla natura bounded, FIFO circolare (con push e pop efficienti), bloccante e chiudibile è ottima per questo framework.

```
typedef struct {
    mr_line_item_t *items;
    size_t capacity;
    size_t head;
    size_t tail;
    size_t count;
    int closed;
    mtx_t lock;
    cnd_t not_empty;
    cnd_t not_full;
} mr_line_queue_t;
```

7 Protocollo sulle pipe

La struttura delle pipe è già descritta nella sezione dei processi; qui vengono discussi i protocolli di serializzazione per una comunicazione sicura e robusta.

Dato che non possiamo trasmettere i puntatori ai dati direttamente (i processi hanno spazi di indirizzamento diversi), ogni messaggio è composto da un header che precede la trasmissione dei byte effettivi.

Le letture e le scritture utilizzano due funzioni helper fondamentali per la robustezza: `readn()` e `writen()`. Queste due funzioni si assicurano che il numero di byte letti/scritti sia quello che ci aspettiamo.

- La prima pipe comunica un header che contiene la lunghezza del nome del file, il numero di riga e la lunghezza della riga, prima di comunicare il nome del file e la riga (come byte grezzi).
- La seconda pipe comunica un header che contiene la lunghezza del token e la lunghezza del valore, prima di comunicare il token (stringa C) e il valore (come byte grezzi).
- La terza pipe comunica un header che contiene la lunghezza del token e la lunghezza del risultato, prima di comunicare il token (stringa C) e il risultato (come byte grezzi). Concettualmente è la stessa struttura usata nella seconda pipe, ma per chiarezza ne è stata creata una seconda versione.

7.1 Gestione dell'ownership

Dato che i puntatori non possono essere trasmessi, è importante definire chi possiede i vari dati durante la pipeline della computazione.

Per ogni trasferimento il mittente mantiene la proprietà sui propri buffer, liberabili dopo la scrittura riuscita. Il destinatario deve invece allocare nuovi buffer per acquisire i dati ricevuti, mantenendo poi la responsabilità della loro liberazione.

Questo passaggio è visibile nella lettura delle righe: `read_line_record()` alloca nuovi buffer per `file_name` e `line`, che poi vengono liberati con `line_item_destroy()` dopo l'elaborazione del mapper.

Lo stesso principio vale per le coppie intermedie. `read_pair_record()` alloca token e valore nel processo reducer; durante il raggruppamento, `pair_groups_add_pair()` trasferisce questi buffer dentro la struttura dei gruppi. Alla fine della computazione, `pair_groups_destroy()` e `pair_group_destroy()` liberano token, valori e risultati collegati ai gruppi.

Così facendo, ogni processo gestisce la propria memoria all'interno del rispettivo spazio di indirizzamento, trasferendo non i puntatori ma il contenuto dei dati lungo la pipeline.

8 Raggruppamento per token

Il raggruppamento per token funziona grazie a una serie di struct che permettono di collegare un token alla lista dei suoi valori e ai suoi risultati, per unificare l'elemento e facilitare la gestione della memoria.

La struttura dei gruppi contiene un array di elementi, insieme a un contatore di entry valide e una capacità massima incrementabile tramite `realloc`.

```
typedef struct {
    mr_pair_group_t *items;
    size_t count;
    size_t capacity;
} mr_pair_groups_t;
```

Ogni gruppo contiene il token con la sua lunghezza, un array di valori con rispettivi contatori e capacità, e un array di risultati (gestito nella stessa maniera, ogni entry è una coppia `<token, data>` con rispettive lunghezze).

```
typedef struct {
    char *token;
    size_t token_len;
    mr_value_t *values;
    size_t values_count;
    size_t values_capacity;
    mr_result_list_t results;
} mr_pair_group_t;
```

Nel processo reducer il raggruppamento avviene prima dell'avvio dei worker thread reducer. Il reducer legge dalla pipe tutte le coppie prodotte dal mapper; per ogni coppia cerca se esiste già un gruppo con lo stesso token (`pair_groups_find()`). Se il gruppo esiste, il valore viene aggiunto all'array dei valori di quel gruppo (`pair_group_add_value()`). Se il gruppo non esiste, viene creata una nuova entry in `mr_pair_groups_t` e il valore viene inserito nel nuovo gruppo (`pair_groups_push_group()`).

Quando la pipe mapper-reducer raggiunge EOF, il reducer sa che non arriveranno altre coppie e quindi il raggruppamento è completo. A quel punto i gruppi vengono ordinati lessicograficamente per token, così l'output finale rimane deterministico. Solo dopo questo passaggio vengono creati i worker thread reducer, che ricevono i gruppi da elaborare e invocano la callback reducer una sola volta per ogni token distinto.

9 Formato del file di output

Il file in output ha formato binario e viene prodotto dal processo principale dopo aver letto i risultati dalla terza pipe con `write_reducer_results()`.

Non è altro che una lista di entry con la seguente struttura:

```
[token_len][result_len][token bytes][result bytes]
[token_len][result_len][token bytes][result bytes]
...
```

Il determinismo del risultato è garantito dall'ordinamento lessicografico effettuato dopo il raggruppamento (prima di chiamare la callback reducer). L'emissione di più risultati non crea problemi: l'ordine è definito dalla callback reducer, quindi non cambia fra le varie esecuzioni e non è responsabilità del framework.

Il risultato è sempre trattato come dato opaco, il framework non lo interpreta.

10 Sistema di log

Il sistema di log serve a tracciare gli eventi principali della computazione, mantenendo separata la diagnostica dal protocollo usato sulle pipe standard. Questo è importante perché nei processi mapper e reducer lo standard output è riservato alla comunicazione interna del framework.

Il file di log è configurabile tramite `mr_attr_set_log_file()`. Se l'utente non specifica un file, viene usato un nome di default (`mr.log`).

Come specificato dal contratto del progetto, ogni riga di log contiene un timestamp, processo, pid, thread, tipo di evento e messaggio:

```
[timestamp] [processo:pid] [thread] [evento] messaggio
```

Gli eventi registrati includono la creazione delle pipe, la creazione dei processi figli, l'apertura e chiusura dei file, avvio e terminazione dei thread, conteggi finali ed eventuali errori.

La gestione dei log è centralizzata nel processo main. Il main scrive direttamente sul file di log, mentre i processi mapper e reducer scrivono su pipe dedicate. Nel processo main vengono creati due thread collector, uno per il mapper e uno per il reducer, che leggono da queste pipe e riportano i messaggi nel file di log.

La scrittura sul file di log è protetta da un lock, così da evitare interferenze fra il thread principale e i thread collector. Nei processi figli è presente un lock locale per sincronizzare le scritture prodotte dai diversi thread dello stesso processo.

11 Esempio applicativo: word count

Come esempio di applicazione è stato scelto un classico contatore di parole. Nel file `examples/word_count.c` viene definito:

- callback mapper: data una riga, viene cercato un token ed eventualmente emesso in coppia col valore 1 (intero);
- callback reducer: dato il token e i suoi valori, vengono accumulati nel risultato (somma) ed emessi;
- funzione di run: per inizializzare, eseguire e controllare l'esecuzione;
- main: per controllare la validità degli input ed eseguire la funzione di run.

Per lanciarlo basta eseguire nel terminale:

- `make example` per compilare il framework e l'esempio;
- `./build/word_count examples/input.txt examples/output.mro examples/word_count.log` per eseguire.

12 Test automatici

I test automatici sono stati scritti per verificare sia il contratto pubblico del framework sia il comportamento end-to-end della pipeline. Lo scopo generale è controllare che la libreria sia usabile da un programma esterno tramite `mr.h`, che gestisca correttamente input, output e callback, e che il risultato non dipenda dall'ordine di scheduling dei thread.

Durante lo sviluppo sono stati scritti dei piccoli test per verificare aspetti più granulari del framework, ma sono stati sostituiti dai tre presenti nella consegna, che sono stati scritti a progetto finito per gli ultimi aggiustamenti.

12.1 Test su attributi e ciclo di vita

Questo gruppo di test verifica la gestione degli attributi e dell'handle principale della computazione. Vengono controllati i valori di default, i setter degli attributi, il rifiuto di parametri non validi e la corretta impostazione di `errno`. Inoltre viene verificato il ciclo di vita dell'oggetto `mr_t`: creazione con callback valide, salvataggio di mapper, reducer e `user_arg`, distruzione finale e gestione dei casi di errore.

12.2 Test end-to-end generico

Il test end-to-end generico verifica l'intera pipeline su un singolo file di input. Usa mapper e reducer artificiali, diversi dal word count, per controllare che il framework non dipenda da una logica applicativa specifica. In particolare verifica che valori binari opachi, valori di lunghezza zero, raggruppamento per token e formato binario dell'output vengano gestiti correttamente. Il test controlla anche che i risultati siano scritti nell'ordine atteso e che non vengano prodotti record extra.

12.3 Test su directory, concorrenza e log

Questo test verifica l'elaborazione di una directory contenente più file, con più thread mapper e reducer e una coda interna di dimensione ridotta. In questo modo viene esercitata la sincronizzazione tra reader e worker e viene controllato che l'output resti deterministico anche in presenza di concorrenza. Il test verifica anche che il log venga prodotto e che contenga eventi significativi come creazione delle pipe, creazione dei processi, avvio dei thread e conteggi finali.

13 Utilizzo di coding agents nel progetto

Anche se non è specificato, credo necessario spendere un paragrafo per descrivere l'utilizzo di strumenti automatici di supporto alla programmazione, che come puntualizzato nel testo "non solleva lo studente dalla responsabilità di comprendere, verificare e saper spiegare il codice consegnato".

Nota: questa relazione è stata scritta senza l'utilizzo di tali strumenti.

Durante lo sviluppo è stato utilizzato il modello `gpt-5.5` con ragionamento "Alto" all'interno dell'applicazione desktop "Codex".

Per garantire un comportamento consono da parte del modello, è stato fornito il seguente file `AGENTS.md` e una versione in markdown del testo originale del progetto.

```
# AGENTS.md
```

```
SEI IN MODALITÀ GUIDA: non scrivere codice in autonomia. Dai per scontato di non dover scrivere codice ma di dover aiutare il programmatore a comprendere al massimo e sviluppare competenze in C.
```

```
## Regole non negoziabili
```

- NON generare mai codice in autonomia. Genera codice solo su richiesta esplicita del programmatore.
- Prima di generare codice, assicurati con domande mirate la comprensione di ciò che sta venendo implementato.
- Lo scopo di questo progetto è imparare la programmazione C in un contesto più avanzato della semplice applicazione dei concetti.
- Non aggiungere commenti al codice in autonomia. I commenti vengono aggiunti dal programmatore.

Ambiente

- Il testo del progetto si trova dentro 'docs/Testo.md', quella è la fonte di verità in qualsiasi contesto. Solo progetto base, niente addendum.
- Il progetto viene sviluppato all'interno di un dev container che simula una macchina con Ubuntu 24.04, cioè il sistema in cui verrà eseguito il codice prodotto alla fine.
- Le best practices dello sviluppo UNIX sono favorite, soprattutto la modularità delle funzioni.

Come si può intuire dal system prompt, l'agent è stato usato principalmente per:

- refattorizzazione guidata del codice;
- aiuto nella comprensione del testo da un punto di vista concettuale e logico;
- scrittura automatica di porzioni di codice ripetitive o comunque non fondamentali da un punto di vista progettuale (setter, controlli su input, strutture simmetriche);
- discussione delle scelte implementative;
- mantenimento del codice e della struttura coerente nel tempo, per evitare deragliamenti di stile che vanno al di fuori della formattazione (quella è stata gestita con un semplice file `.clang-format` e un formatter automatico all'interno dell'editor).

Questo sistema mi ha permesso di non delegare il ragionamento dietro al codice, per garantire una comprensione del progetto e del codice scritto mantenendo una velocità di sviluppo moderata nel tempo.

14 Conclusioni

Il progetto realizza un piccolo framework MapReduce locale in linguaggio C, utilizzabile da programmi esterni tramite l'interfaccia pubblica `mr.h`.

Gli aspetti più delicati sono stati la gestione corretta dei descrittori, la propagazione dell'EOF, la serializzazione dei dati fra processi, l'ownership della memoria e la sincronizzazione dei thread. Il framework tratta i valori come dati opachi, quindi non dipende da un esempio specifico.

I test automatici e il sistema di log sono stati usati per controllare il comportamento della libreria e rendere più semplice l'individuazione degli errori. Nel complesso il progetto ha permesso di applicare concetti tipici della programmazione UNIX e concorrente in un contesto più strutturato rispetto a un singolo programma sequenziale.